Q1. Why did Java 8 introduce default methods?

Java 8 introduced default methods to allow interfaces to evolve without breaking existing implementations. They provide default behavior so new methods can be added while preserving backward compatibility. For example, interfaces such as Collection introduced default methods like forEach().

Q2. Default methods vs static methods in interfaces

Default methods are invoked on interface instances and can be overridden by implementing classes. Static methods belong to the interface itself and cannot be overridden. Default methods extend interface behavior, while static methods act as utility helpers.

Q3. Functional Interface and @FunctionalInterface

A functional interface has exactly one abstract method and serves as the foundation for lambda expressions. The @FunctionalInterface annotation ensures this constraint at compile time. Functional interfaces may still contain multiple default methods.

Q4. Four major functional interface categories

Consumer consumes input without returning a value, Supplier provides a value without input, Predicate evaluates a condition and returns a boolean, and Function transforms input to output.

Q5. Effectively final variables

A variable is effectively final if it is assigned once and never modified. Java enforces this rule for lambda expressions to ensure consistent behavior and thread safety.

Q6. Method reference types

Method references include ClassName::staticMethod, object::instanceMethod, and ClassName::instanceMethod. In the third form, the first parameter of the lambda becomes the target object of the instance method.

Q7. Optional.of() vs Optional.ofNullable()

Optional.of() does not accept null values and throws a NullPointerException if null is passed. Optional.ofNullable() safely handles null values and should be used when null is possible.

Q8. orElse() vs orElseGet()

orElse() always evaluates its argument, while orElseGet() evaluates lazily. orElseGet() is more efficient when the default value is expensive to compute.

Q9. Why Optional.get() is dangerous

Optional.get() may throw NoSuchElementException if the value is absent. Recommended alternatives include orElse(), orElseGet(), ifPresent(), and orElseThrow().